

Observabilité d'un jeu multijoueur

Rattrapage UE03 / 6 octobre 2026

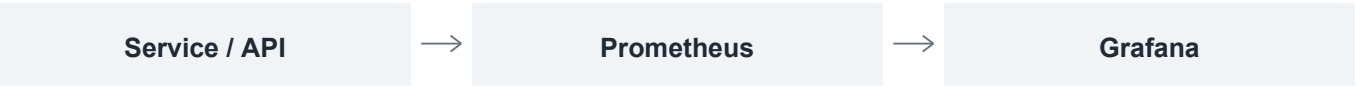
Ce projet réunit le service de télémétrie fourni, une chaîne de collecte, trois tableaux de bord Grafana et une CI qui bloque la publication en cas d'échec.

Le principal incident concerne le rendu de l'overlay

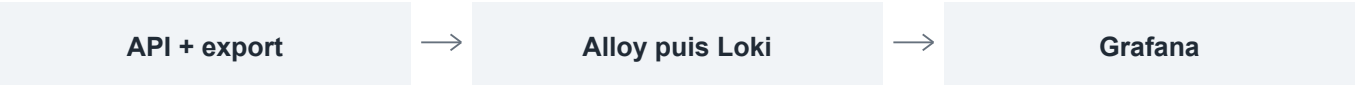
À partir de la build beta-20260924-3, **61,59 % des rapports cohérents** de la cohorte haute résolution avec bloom portent une signature overlay, contre moins de 1 % auparavant. Hors de cette configuration, la proportion reste autour de 1 %. Le coût se concentre dans le travail CPU de l'overlay.

Constat	Résultat
Qualité de l'export	2 889 événements uniques après retrait de 55 doublons.
Rapports incohérents	70 rapports exclus des statistiques de performance.
Victoires très courtes	83 parties vault, dont 14 reliées à des rapports clients.
Retards de bots	91 rapports dans 33 parties foundry.

Architecture



Métriques HTTP et alertes - collecte toutes les 10 s



Événements JSONL - détails des rapports et des parties



Tri, déduplication et conservation des lignes sources

Les métriques suivent le service en cours d'exécution. Les logs historiques conservent leurs dates d'origine et servent à l'enquête. Les proportions présentées dans le rapport concernent les pics reçus, pas l'ensemble des joueurs.

Qualité de l'export

Chaîne d'import : `src/export-parser.js`, `scripts/import-history.js` et `scripts/analyze.py`

Contrôle	Résultat observé	Traitement
Structure	2 944 blocs ; aucun JSON rejeté	Séparation par en-têtes, reconstruction du JSON multiligne ; BOM/CRLF acceptés.
Doublons	55 exacts : 38 rapports, 17 fins	Clé type + identifiant ; empreinte JSON canonique. Une collision est inscrite dans l'audit avec sa ligne source ; le second événement est exclu du JSONL et l'import sort en erreur.
Horodatage	20/09 00:31:37 → 27/09 00:04:13 Paris	Conversion explicite Europe/Paris → UTC ; tri croissant pour Loki. Six événements après minuit sont conservés.
Ordre	Deux inversions locales : 1 s et 3 s	Ligne source conservée ; tri indépendant de l'ordre décroissant de l'export.
Schéma	2 085 v1 et 25 v2	RTT racine en v1, <code>network.rttMs</code> en v2 ; normalisation sans perte.
GPU absent	882 valeurs nulles / 2 110	Toutes avec <code>gpuTimerSupported=false</code> . Aucune conversion en zéro.
Cohérence métier	70 rapports invalides / 2 110 (3,32 %)	Temps de stages, détails et sommes vérifiés ; causes et issues conservées.

Ce que la fenêtre ne permet pas d'affirmer

Le plus grand intervalle sans événement va du 22 septembre 13:59:23 à 16:14:34, soit **2 h 15 min 11 s**. Il peut refléter une perte de collecte ou une absence d'événements exportés : sans heartbeat, ce trou ne prouve pas une indisponibilité. Quatre rapports cohérents de deux parties n'ont pas de fin correspondante, en plus des 70 rapports invalides.

Les 25 rapports v2 ajoutent physics sans l'intégrer à totalMs dans le support fourni (écart de 0,4 à 1,4 ms). Tous les stages sont validés, mais physics est exclu de cette somme. La tolérance de 3 ms absorbe les arrondis ; elle ne rend pas acceptable une durée négative.

Requêtes SQL : `volumes`, `audit`, `quality_issues`, `time_range`, `daily_counts`, `export_order`, `schema_rtt`, `gpu_missing`, `largest_gaps`, `missing_completions`, `v2_stage_difference`, `invariants`. Fichier : `docs/queries.sql`.

Empreinte SHA-256 de l'export source (6 072 091 octets, 264 344 lignes) :

```
4e4679501817517615eb5f9521fd9ff8a09fb
256009ddb8794c49b8d34ee0e1b
```

Classification des rapports

Les mêmes fonctions enrichissent l'historique et les événements en direct.

La classification attribue une cause principale avec un ordre explicite : invalid → hidden → overlay → shader → world → network → render → unknown. Les signatures secondaires restent disponibles ; les détections de bots périmés et de victoires courtes sont indépendantes.

Cause / volume	Signature retenue	Précaution
invalid · 70	Type, finitude, signe et cohérence des temps non respectés.	Une instrumentation défectueuse peut produire le même signal qu'une falsification.
hidden · 158	visibilityHidden et délai navigateur ≥800 ms.	Évite d'attribuer le throttling d'un onglet masqué au GPU.
overlay · 262	renderOverlay ≥200 ms.	Chercher la cohorte résolution/bloom ; le seuil seul ne prouve pas une régression.
shader · 459	Première capture et ≥40 nouveaux programmes.	Compilation initiale probable ; ce coût peut être attendu lors d'un chargement.
world · 20	worldDynamics ≥60 ms.	Les 20 cas appartiennent ici à la cohorte bots périmés.
network · 92	reason=network et RTT ≥180 ms ou intervalle maximal entre ticks ≥150 ms.	Un RTT élevé ne distingue pas à lui seul client, réseau et serveur.
render · 1 049	Stage render positif sans signature précédente.	Catégorie résiduelle, pas diagnostic causal complet.
unknown · 0	Aucune signature reconnue.	Prévue pour éviter de forcer tous les futurs cas dans une classe connue.

Seuils et contrôles

Les seuils de classification sont dans config/detection.json ; les seuils bots (2 bots, 1 000 ms) restent dans enrich(), et les seuils opérationnels dans rules.yml. Ils séparent les groupes visibles dans le support : coût overlay de plusieurs centaines de millisecondes, forte création de shaders et retards de bots de plusieurs secondes. Ils doivent être recalibrés sur une production réelle : cet export ne contient que les pics remontés ; le service fourni sert à simuler la démo.

Les tests altèrent notamment physics, renderOverlay et les sommes des détails pour s'assurer qu'une valeur négative, une chaîne numérique ou un coût impossible ne traverse pas la validation. Les 70 rapports invalides restent consultables ; ils ne polluent pas l'histogramme des frames.

Requêtes SQL : causes, quality_issues, bots, bot_comparison. Fichier : docs/queries.sql.

Métriques et collecte

Scraping toutes les 10 s ; règles évaluées toutes les 15 s.

Métrique	Type et justification	Labels
game_http_requests_total	Counter : nombre de réponses terminées. Débit via rate ; codes 5xx pour les erreurs.	method, route, status
game_http_duration_seconds	Histogram : distribution en secondes ; p95 calculable et agrégable. Buckets de 5 ms à 2 s.	method, route
game_active	Gauge : état instantané qui monte et descend avec les parties actives.	Aucun
game_reports_total	Counter : inclut invalid et hidden pour mesurer aussi la qualité.	source, build, cause
game_frame_duration_seconds	Histogram : frames signalées, hors invalid/hidden/unknown. Buckets de 16 ms à 5 s.	source, build, cause
game_completed_total / game_short_wins_total	Counters : fins de partie et victoires courtes ; durée corrigée par SPEED dans la simulation.	map / aucun
game_server_gap_reports_total / game_stale_bot_reports_total	Counters : rapports cohérents avec dégradation serveur ou décisions de bots périmées.	source / source, map

Les sources sont limitées à simulation/ingest ; les cinq builds connues plus other et les huit causes bornent le compteur de rapports à 96 séries. Les routes inconnues deviennent unmatched. Aucun identifiant client, serveur ou URL arbitraire n'est un label Prometheus. Les séries de détection sont initialisées à zéro pour que la première rafale puisse être comptée.

Trois recording rules

```
game:api_request_rate:5m
game:api_error_ratio:5m
game:api_latency_p95:5m
```

Le débit exclut /healthz. Le ratio 5xx revient à zéro seulement si une série de trafic existe ; une métrique absente reste absente. Le p95 utilise histogram_quantile sur les buckets agrégés : son exactitude dépend de leur largeur. /metrics ne compte pas sa propre lecture.

Index Loki et conservation

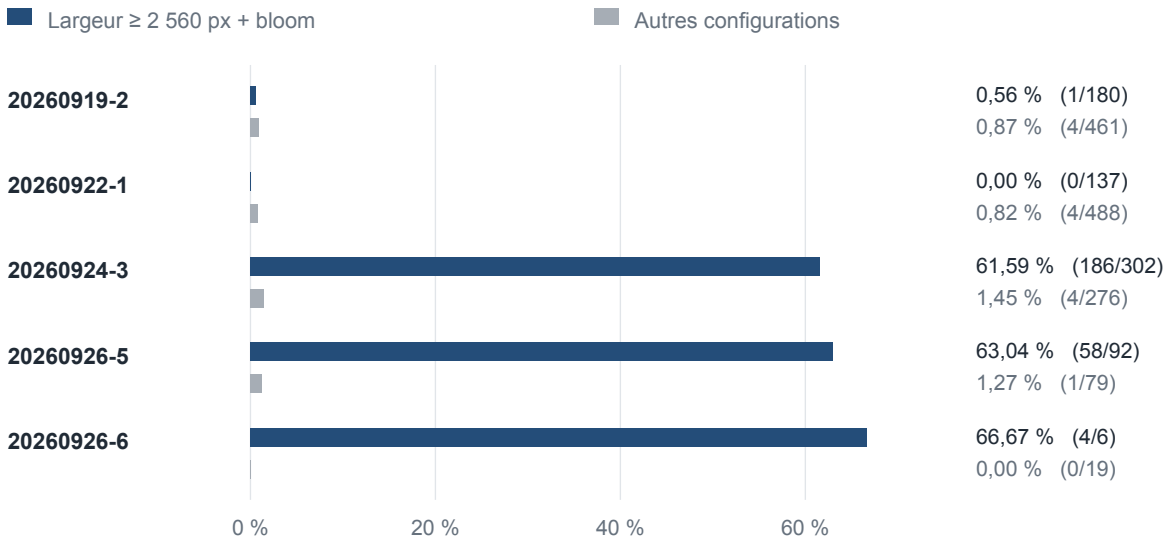
Seuls service_name, dataset, event et source sont indexés. Les champs détaillés sont extraits à la requête. Alloy conserve ses positions et lit le JSONL sans socket Docker. L'historique utilise ses timestamps originaux ; les derniers blocs sont envoyés après 30 s. Un import identique préserve le fichier et son inode. Un autre export doit utiliser un jeu de données distinct pour éviter de réimporter des événements dans le même flux.

Références : documentation Prometheus sur les histogrammes et Grafana Loki sur la cardinalité (liens p. 10).

Régression de l'overlay

Population étudiée : rapports cohérents, largeur $\geq 2\,560$ pixels et bloom actif.

Part des rapports avec une signature overlay



Une rupture à partir de beta-20260924-3

La cohorte passe de 1/180 puis 0/137 rapports overlay à **186/302** sur -3. La régression persiste sur -5. Hors cohorte, les proportions restent entre 0 et 1,45 %. Avec seulement six rapports dans la cohorte, la build -6 ne permet pas de conclure à une amélioration ou à une aggravation.

Un coût surtout CPU

Sur les **249 rapports overlay** de cette cohorte, le coût médian de renderOverlay est de **434 ms** et sa part médiane du travail CPU de **90,4 %**. Le RTT médian est de 34,8 ms et l'intervalle maximal moyen entre ticks avoisine 51 ms dans les cinq familles de navigateurs. Une panne réseau générale explique mal ces résultats.

Seules 118 observations disposent d'une mesure GPU, âgée de 107,6 à 884,5 ms. Elle ne décrit donc pas nécessairement la frame courante.

Exemple : P-41a28e20-18, ligne 106142, le 24 septembre à 20:16:26. Largeur 3 440 px et bloom actif. Frame de 523 ms, dont 471,1 ms pour l'overlay. RTT de 58,4 ms et intervalle maximal entre ticks de 50,96 ms.

Requêtes SQL : build_cohorts, overlay_population, overlay_context, overlay_evidence. Fichier : docs/queries.sql.

Le nombre total de frames et de joueurs exposés manque. Plusieurs rapports d'un même client ne constituent pas autant de joueurs affectés.

Postmortem : overlay

Incident étudié rétrospectivement ; toutes les heures ci-dessous sont en Europe/Paris.

Date / heure	Fait vérifiable	Lecture
20/09 → 24/09	9 signatures overlay sur les deux anciennes builds.	Il existe un bruit de fond ; l'overlay n'apparaît pas pour la première fois jeudi.
24/09 · 09:35:45	Premier rapport -3, mais incohérent.	Exclu pour dater la diffusion effective d'une build.
24/09 · 19:06:36	Premier rapport cohérent -3.	Première observation, pas preuve de l'heure de déploiement.
24/09 · 20:16:26	Premier overlay -3 ; exemple p. 5.	Début observable de la signature sur cette build.
26/09 · 12:32:23	Premier rapport cohérent -5.	59 overlays au total sur -5 ; cohorte encore à 63,04 %.
26/09 · 20:50:28	Premier rapport cohérent v2 / -6.	4 overlays dans la cohorte de 6 ; pas de preuve de rétablissement.

Cause probable et impact

L'hypothèse la mieux étayée est un chemin CPU d'overlay devenu coûteux avec la grande résolution et bloom à partir de -3. La concentration par configuration, le contraste avec les autres rapports et la décomposition du travail convergent. Le générateur fourni reproduit volontairement cette condition ; il illustre ce mécanisme possible, sans établir ce qui a été déployé dans une véritable release cliente.

La population couvre Brave, Chrome, Edge, Firefox et Safari. Les 249 rapports de la cohorte portent un coût overlay de plusieurs centaines de millisecondes ; cela suffit à provoquer des saccades sévères. L'export ne permet pas de calculer un nombre de joueurs uniques réellement affectés ni une perte de disponibilité globale.

Actions proposées, à valider par une nouvelle mesure

Priorité	Action	Critère de validation
Immédiate	Tester la désactivation de l'overlay/bloom dans la cohorte ; conserver une option de retour.	Comparer le coût CPU et la fréquence des pics sur un échantillon contrôlé.
Corrective	Profiler le chemin overlay à 2 560/3 440/3 840 px ; comparer les builds et limiter les recalculs.	Mesure avant/après sur mêmes scènes et matériel, sans déplacer le coût vers un autre stage.
Préventive	Ajouter tests de budget frame et canary par résolution/bloom.	Alerte stable sur un volume suffisant ; pas de régression sur les autres configurations.

Requêtes SQL : build_first_valid, overlay_timeline, overlay_evidence, overlay_context. Fichier : docs/queries.sql.

Intégrité et incidents distincts

Trois groupes à examiner séparément : victoires courtes, bots en retard et rapports incohérents.

Victoires rapides : un signal de farming à investiguer

83 parties vault, toutes 3–0, durent entre 54,349 et 74,430 s : 45 le 23 septembre entre 01:14:22 et 03:46:35 ; 38 le 25 entre 00:44:29 et 02:32:54. Seules 17 sont marquées en quarantaine ; 66 ne le sont pas. Le seuil de 100 s isole cette répétition sans confondre les parties foundry de près de trois à cinq minutes.

Les jointures relient 14 parties à trois clientId : 7247704e (7), 5db9ad62 (5), 7440b319 (2). Les 69 autres n'ont pas de rapport client associé dans cet export. Attribuer les 83 parties à ces trois identifiants dépasserait les données. Il faut vérifier côté serveur adversaires, actions et conditions de victoire.

Bots périmés : une explication concurrente aux victoires faciles

33 parties foundry (13 le 21 septembre, 20 le 24) concentrent **91 rapports** issus de 32 clients distincts. Deux ou trois bots ont des décisions âgées de 2 058 à 8 992 ms, contre 18–45 ms ailleurs. Les parties sont toutes 3–0 et durent de 170,815 à 282,434 s. Les 20 signatures world appartiennent à ce groupe.

La cohérence serveur/carte/retard de décision soutient un problème de traitement des bots. Gagner contre ces bots ne démontre pas une manipulation du client. L'investigation doit comparer charge du serveur, délai de décision et temps worldDynamics.

Falsification probable ou instrumentation gravement défectueuse

Le client **5e1f0c7a** envoie 70 rapports associés à 70 identifiants serveur, du 24 septembre 09:35:45 au 26 à 22:37:37. Ils affichent jusqu'à 1 200 FPS, un stage render pouvant descendre à –101,5 ms et des sommes incompatibles. Ces valeurs sont physiquement incohérentes : les exclure est justifié, mais l'intention de fraude ne peut être prouvée par le contenu seul.

Dégradation serveur autour de 03 h

16 rapports cohérents présentent un tickGapMaxMs ≥ 150 ms, concentrés le 21 septembre entre 02 h et 03 h, puis le 23 autour de 03 h. Les maxima atteignent environ 747 ms. Comparer ces événements aux RTT aide à distinguer un blocage serveur d'un simple pic réseau client. Tous les 92 rapports classés network ne relèvent pas de cet épisode.

Suites correctives et préventives proposées

À l'ingestion, isoler les rapports incohérents et vérifier le schéma avant toute utilisation métier. Pour le farming, valider les résultats côté serveur et tracer le rattachement partie/client ainsi que les actions de jeu ; la vitesse seule ne doit pas déclencher de sanction. Pour les bots, profiler la file de décisions puis suivre son âge, avec un test de charge qui vérifie leur progression.

Requêtes SQL : short_wins, short_win_clients, bots, bot_comparison, invalid_clients, server_degradation. Fichier : docs/queries.sql.

Alertes et seuils

Règles complètes dans config/prometheus/rules.yml ; tests dans tests.yml.

Alerte	Condition / maintien	Faux positifs ou négatifs
TelemetryUnavailable	up=0 pendant 1 min.	Redémarrage volontaire ; ne surveille pas une cible retirée de la configuration.
ApiErrors	5xx >5 %, débit >1 req/s ; 2 min.	Évite le bruit à faible volume ; peut manquer une panne très peu sollicitée.
ApiLatency	p95 >250 ms, débit >1 req/s ; 2 min.	Rafale locale ou CPU partagé ; un pic bref peut rester sous le délai.
OverlayRegression	Overlay >20 % des rapports hors invalid/hidden sur 5 min, au moins 20 rapports ; 2 min.	Échantillonnage des pics et charge synthétique ; manque une cohorte rare.
InvalidTelemetry	≥3 rapports invalides en 5 min ; 1 min.	Un bug de schéma peut ressembler à une falsification.
ShortWins	≥5 victoires vault 3–0 <100 s en 5 min ; 1 min.	Parties légitimes ou tests ; farming plus lent non détecté.
ServerTickDegraded	≥5 rapports cohérents avec tickGapMaxMs ≥150 ms sur 5 min ; 1 min.	Plusieurs rapports d'une même partie ; incidents sans télémétrie invisibles.
StaleBotDecisions	≥5 rapports cohérents, ≥2 bots et âge ≥1 000 ms sur 5 min ; 1 min.	Même serveur surreprésenté ; bots dégradés sans rapport client invisibles.

Pourquoi ces valeurs ?

Les délais d'une à deux minutes filtrent un scrape isolé et les transitions. Les fenêtres de cinq minutes amortissent les rafales. Le minimum de 20 rapports évite qu'un seul overlay produise artificiellement 100 %. Le seuil de 20 % est un point de départ : il se situe loin du bruit historique hors cohorte, mais n'est pas un SLO contractuel.

Les seuils 100 s, 1 000 ms et 150 ms séparent des groupes observés : victoires de 54–74 s, bots au-delà de deux secondes, intervalles entre ticks de plusieurs centaines de millisecondes. Une calibration sur trafic complet reste nécessaire.

Validation et portée opérationnelle

21 scénarios promtool vérifient les délais, les limites, les faibles volumes, la séparation par source/build, les absences de données et les recording rules (32 assertions d'alertes et 8 assertions PromQL). Les alertes sont consultables dans Prometheus et Grafana ; aucune notification externe n'est configurée.

Le loadgen provoque environ 25 % d'overlays parmi les rapports hors onglets masqués. Une OverlayRegression active pour source=ingest est donc attendue dans la démo. Elle permet de vérifier le déclenchement de l'alerte sous charge locale.

Construction et CI

Validation locale. Publication GitHub en attente.

Une image réduite et un déploiement reproductible

Le Dockerfile installe les dépendances dans une étape dédiée et ne garde en exécution que les dépendances de production, src et config. La base Node 22 Alpine est épinglée par digest. Le processus tourne en utilisateur node ; Compose impose une racine en lecture seule, retire les capacités et limite les ports à 127.0.0.1. Les sept services démarrent par docker compose up --build ; history est un import ponctuel qui sort normalement avec le code 0.

La publication dépend de tous les contrôles

```
lint -> tests -> import/analyse -> promtool  
-> build -> scan CRITICAL -> GHCR : sha-<commit complet>
```

Le job image dépend du succès de validate. Le scan utilise une archive de l'image, sans exposer le socket Docker au scanner. Trivy sort avec le code 1 pour une vulnérabilité CRITICAL. La publication GHCR est limitée à main, hors pull request et hors scénario de démonstration. Les actions et images sont épinglées.

Contrôle exécuté	Résultat	Preuve livrée
Lint + tests Node	18 tests réussis ; 0 échec	evidence/tests.log
Règles Prometheus	21 scénarios réussis	evidence/promtool.log
Stack et dashboards	2 110 + 779 logs ; 28 requêtes valides ; collecte live active	evidence/stack-verification.json
Échec de test contrôlé	UE03_DEMO_FAIL=1 : code 1	evidence/ci/test-failure.log et .exit
Image de démo vulnérable	lodash 4.17.11 : CVE-2019-10744 CRITICAL, code 1	evidence/ci/trivy-vulnerable.json
Image applicative normale	Aucune CRITICAL détectée ; code 0	evidence/ci/trivy-normal.json

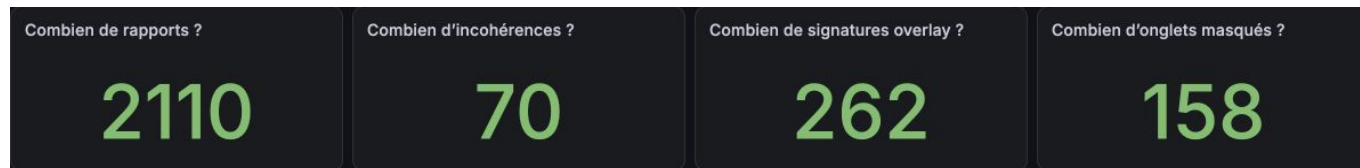
La dépendance vulnérable est ajoutée uniquement à une image jetable de démonstration ; elle n'entre pas dans le service livré. CVE-2019-10744 est corrigée à partir de lodash 4.17.12 selon le rapport du scanner. Un scan sans CRITICAL n'est pas une garantie d'absence de toute vulnérabilité ; il dépend de la base consultée et du périmètre analysé.

Tableaux de bord et reproduction

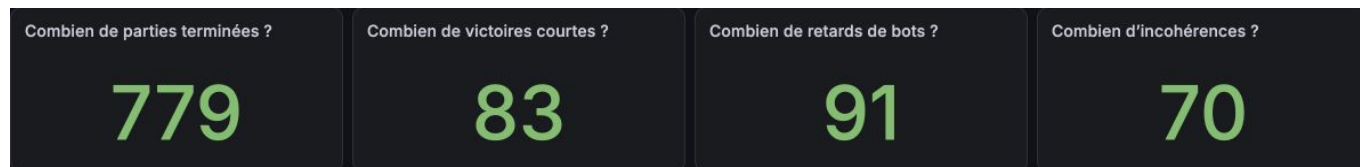
Trois dashboards provisionnés dans le dossier UE03 Observabilité de Grafana.



Santé du service : disponibilité, débit, p95, erreurs HTTP et parties actives.



Performance : volumes historiques, incohérences, overlays et onglets masqués.



Activité et intégrité : fins de partie, victoires courtes et retards de bots.

Relancer les contrôles

```
docker compose up -d --build
npm ci --ignore-scripts
npm run verify
npm run import:history && npm run analyze
python3 scripts/verify-stack.py
```

Grafana : localhost:3000. Le compte local est indiqué dans le README. Les liens historiques règlent la période et le jeu de données. Les 25 requêtes SQL sont dans docs/queries.sql. Les captures complètes et sorties de tests sont dans evidence/.

Limites du déploiement

La stack fonctionne sur une seule machine, sans haute disponibilité ni notification externe. Les métriques en direct proviennent du générateur fourni. L'analyse ne démontre ni l'intention d'un joueur ni le rétablissement de la régression.

Références techniques

Prometheus : histogrammes

Prometheus : tests unitaires des règles

Grafana Loki : cardinalité des labels

Trivy : code de sortie du scanner